AI Resume Screener & Recruiter Hub

Comprehensive Technical Reference & Interview Preparation Guide

---

Candidate Name: Vishal Khorwal

Target Role: Software Engineering Intern

Academic Qualification: B.Tech. in Computer Science & Engineering (2023 - 2027)

Application Stack: FastAPI, React, SQLite, Scikit-Learn, PyMuPDF, ReportLab

Project Context: The AI Resume Screener is an enterprise-grade recruitment assistant designed to streamline candidate evaluation by scoring resumes against dynamic job descriptions. This document acts as an exhaustive architectural blueprint and technical preparation resource, explaining the development process, codebase file structure, dynamic scoring algorithms, and complex engineering decisions solved during deployment.

1. Codebase Directory Structure & File Index

The project is split into a backend REST API directory and a frontend React single-page application (SPA). Below is the comprehensive file index explaining the purpose, imports, and dependencies of every module:

Backend Component Directory (backend/)

- **main.py**: The core FastAPI entry point. Defines all REST routing (e.g. `/auth/login`, `/auth/register`, `/screen`, `/sessions`, `/export/*`, `/health`). Manages CORS middleware settings and imports authentication, database, parser, and scoring helper modules.
- **database.py**: The database management layer. Dynamically connects to PostgreSQL (via DATABASE_URL using the pure-Python pg8000 client with SSL enabled) when deployed in production, or falls back to a local SQLite database in /tmp/resume_screener.db. Manages cross-database query execution, row-to-dictionary conversion, and auto-seeding.
- **auth.py**: The authentication module. Handles password hashing (SHA-256) and JWT token generation, signature checks, and role verification (e.g., verifying user claims to validate permission controls).
- **scorer.py**: The scoring engine. Combines custom keyword matches against a database of technical skills, TF-IDF cosine similarity vectorization using Scikit-Learn, and dynamically scaled experience scores based on JD context.
- **parser.py**: The file parser. Utilizes PyMuPDF (`fitz`) to extract text from PDF binaries, and `python-docx` to read XML structures in Word (`.docx`) uploads, cleaning whitespace and control characters.
- **exporter.py**: The document exporter. Contains ReportLab rendering rules to output PDF session summaries and OpenPyXL formatting code to export spreadsheet results.
- **models.py**: Holds Pydantic validation schemas (`LoginRequest`, `RegisterRequest`) to enforce strict API payloads.
- **requirements.txt**: Declares backend dependencies: `fastapi`, `uvicorn`, `PyMuPDF`, `python-docx`, `scikit-learn`, `reportlab`, `openpyxl`, `python-jose`, `passlib`, `bcrypt`, `pydantic`, `pg8000`.

Frontend Component Directory (frontend/)

- **src/App.jsx**: The React application orchestrator. Houses global states (active user session, auth token) and controls high-level layout routing. Implements three animated blur blobs for the background design.
- **src/api.js**: Centralizes fetch configurations, appending the authorization headers dynamically and resolving server URLs based on environment variables.
- **src/components/Login.jsx**: Renders register/login toggle forms, verifying passwords and processing credentials securely.
- **src/components/Screener.jsx**: The main dashboard. Houses the drag-and-drop file uploader and runs the sequential batch screening loop (slicing arrays in groups of 4 files).
- **src/components/ResultsTable.jsx**: Displays candidate names, final scores, education levels, and experience with sorting capabilities.
- **src/components/StatsCards.jsx**: Shows metrics (Total Screened, Shortlisted Count, Average Score, Top Score, Bias Flags).
- **src/components/SkillsChart.jsx**: Displays HTML5 Canvas charts of skill distributions among candidate resumes.
- **src/components/History.jsx**: Displays past screening sessions queryable from database records.

2. Development Process & Chronology

The project was built through a systematic engineering workflow designed to transition from a prototype to a robust, Vercel-ready, production-grade application:

Step 1: Baseline Prototype

Built the baseline API logic with simple PDF/DOCX text parsing and a mock memory-based user dictionary. Verified basic math models for skills match and similarity scores.

Step 2: Google Modern & Liquid Glass UI Styling

Overhauled the frontend styling using Outfit typography. Designed a premium glassmorphic interface (using blurs, borders, and gradients) and integrated animated backdrop gradient blobs to create visual depth.

Step 3: CORS Resolution for Separate Deployments

Configured CORS middleware to allow credentials and sanitize trailing slashes. Added a regex pattern matcher so that both local localhost development and dynamically generated Vercel branches can securely call the backend.

Step 4: Ephemeral Persistence for Stateless Servers

Recognized that serverless platforms possess read-only directories, defaulting SQLite databases to the writable `/tmp` path on import. Implemented defense checks to auto-initialize schemas on startup.

Step 5: Client-Side Sequential Batch Uploads

Slices selected file arrays on the frontend in groups of 4. Sends them sequentially to `/screen`, capturing the first returning `session_id` and forwarding it to subsequent chunks to merge candidate lists dynamically, resolving the 10s timeout constraint.

Step 6: Dynamic Experience Scaling for Internships

Recognized student candidates were failing shortlists due to 0 yrs experience. Configured the scoring engine to identify 'internship' keywords in the JD, auto-setting required experience to 0.0 to waive penalties.

Step 7: Cloud PostgreSQL Database Integration for Multi-Device Logins

Integrated support for cloud PostgreSQL databases (like Neon or Supabase) via the DATABASE_URL environment variable. Implemented the pure-Python pg8000 database adapter to resolve stateless file loss in ephemeral serverless containers, enabling persistent recruiter and admin login profiles across different devices and locations.

3. Complete Technical Interview Preparation (20 Q&As;)

Q1: What are the primary constraints of deploying a Python backend on Vercel, and how did you resolve them?

A: Vercel serverless functions have a 10s execution timeout and a 4.5MB request payload limit. To resolve this, I implemented client-side sequential file chunking (batching in sizes of 4) and a state-appending backend endpoint. This prevents gateway timeouts and payload overflow, supporting over 1,000+ files.

Q2: How does the backend database layer persist data given that Vercel containers are stateless?

A: Vercel serverless containers are ephemeral and independent. To resolve this, I implemented a hybrid database adapter. If the environment variable `DATABASE_URL` is set, the app connects to a persistent cloud PostgreSQL database (using the pure-Python `pg8000` client with SSL). Otherwise, it falls back to a local SQLite database at `/tmp/resume_screener.db`. This allows seamless local development with automated cloud persistence in production.

Q3: Explain how your resume scoring engine calculates candidate ranks.

A: The final score is a weighted combination of three components: Skills Match (50% weight), TF-IDF Cosine Similarity (30% weight), and Experience Score (20% weight). Candidate results are sorted in descending order of this final score.

Q4: How does the experience parser work, and how did you prevent student candidates from failing the shortlist threshold?

A: The backend searches the resume text for patterns indicating years of experience. For interns, this yielded 0.0 yrs, costing them 20% of their final grade. I created a dynamic threshold helper: if the Job Description contains words like 'intern' or 'fresher', the target required experience drops to 0.0, awarding the student full points (20/20) for experience.

Q5: Why did you choose TF-IDF vectorization instead of a simple regex matching for the similarity score?

A: TF-IDF (Term Frequency-Inverse Document Frequency) measures not just the presence of keywords, but their relative importance and density across documents. Using Scikit-Learn's TfidfVectorizer with bigrams allows us to capture phrases (like 'Machine Learning' or 'Full Stack') and compute cosine similarities, representing semantic overlap more accurately than simple keyword searches.

Q6: How did you implement secure authentication and authorization?

A: In the SQLite database layer, I hash passwords using SHA-256 before storage. Upon login, the server generates a JWT containing the user's email and role ('Admin' or 'Recruiter') with a sliding expiry. FastAPI dependencies decode and verify the JWT signature on restricted endpoints, filtering database results based on roles.

Q7: How does the CORS setup handle deployment environments vs. local testing?

A: The CORSMiddleware configuration dynamically parses active frontend URLs from environment variables. Additionally, it uses a regular expression pattern to authorize any wildcard local hosts (like `localhost` or `127.0.0.1` with any port) and Vercel preview domain prefixes, preventing CORS errors during staging deployments.

Q8: If a backend serverless function crashes, how does the frontend catch it without throwing CORS errors?

A: Unhandled serverless crashes bypass standard HTTP responses, causing the browser to block the response due to missing CORS headers. I wrapped all endpoints in try-except statements that print the traceback to logs and raise a clean FastAPI HTTPException(500), guaranteeing correct CORS headers are returned so the frontend can display the actual error message to the recruiter.

Q9: Why are PyMuPDF (fitz) and python-docx selected for text extraction?

A: PyMuPDF is a highly optimized C-extension wrapper that extracts text from PDFs 10x faster than pure-python alternatives, while python-docx efficiently handles XML-based Word structures. This minimizes CPU cold starts under Vercel's strict timeout limits.

Q10: If this system scales to handle millions of resumes, what improvements would you implement?

A: I would replace SQLite with a distributed PostgreSQL cluster, offload the text extraction and scoring to a Celery worker queue using Redis, store physical resumes in Amazon S3, and introduce a vector database (like Pinecone) for semantic vector searches.

Q11: Explain how the frontend state management resolves session consistency during batch uploads.

A: In `Screener.jsx`, we maintain `results` and `sessionId` states. During the loop, each resolved promise returns the latest cumulative candidate array and the active session ID. The next index iteration appends this ID to the FormData payload, allowing the backend to retrieve the SQLite session record and append the next batch securely.

Q12: How does the UI design system achieve the 'Liquid Glass' aesthetic?

A: We use dynamic backdrop filters (`backdrop-blur-3xl`), transparent borders (`border-white/[0.08]`), and light opacity backings (`bg-white/[0.03]`). Three high-blur absolute divs with gradients animate using keyframes behind the panels to simulate depth and motion.

Q13: How did you implement user registration role constraints in the database schema?

A: The `users` table schema includes a `role` TEXT field constrained to 'Admin' or 'Recruiter'. When creating a user, the backend validates that the payload input matches one of these pre-approved roles, enforcing strict database domain integrity.

Q14: Explain the difference between the Skills Match Score and the TF-IDF Similarity Score.

A: The Skills Match Score calculates a binary intersection: how many of the required JD skills are present on the candidate's resume (yielding a 0.0 - 1.0 ratio). TF-IDF measures word usage frequency, weighting rare words higher. It calculates the cosine angle between two multi-dimensional term vectors, capturing context and keyword density.

Q15: Why is database auto-initialization called during file imports and startup events?

A: Serverless platforms can spin up new containers to handle concurrent traffic. A container starting up might have an empty `/tmp` directory. Placing `init_db()` in module imports and startup handlers guarantees that database tables and default accounts are dynamically created before any request endpoint executes query statements.

Q16: How do you verify that password hashes match during login?

A: During login, the server hashes the input password using SHA-256 and compares the resulting hexadecimal string against the hashed password stored in the database. If they match, it validates the login.

Q17: Why did you choose OpenPyXL and ReportLab over other spreadsheet/PDF generators?

A: OpenPyXL provides complete control over workbook structures and styling (like cell fills and borders) without third-party CLI tools. ReportLab uses a robust flowable layout engine, rendering custom document flowables dynamically and securely without external server processes.

Q18: What is the purpose of Pydantic models in models.py?

A: Pydantic schemas enforce type safety at the API layer. They automatically validate request structures, parse incoming JSON strings into Python datatypes, and raise clear validation errors (HTTP 422) if fields are missing or typed incorrectly.

Q19: Explain the data lifecycle of a uploaded resume file in the backend.

A: The backend reads the file binary in memory (`await resume_file.read()`), parses the string contents based on file extensions, scores it, and returns the metadata. The document binary is never saved on Vercel's disk, optimizing memory and security.

Q20: How did you ensure security constraints between different recruiters?

A: The JWT includes the user's role and email (as subject). When calling `/sessions`, the database helper filters records by the token's email if the role is 'Recruiter', preventing recruiters from accessing other recruiters' screening logs. Admins bypass this filter.

This document has been pushed to the GitHub repository as requested.